

Pixels to Predictions: DL Vision Challenge

Nidhish Gautam | ng3483

Deeraj Palaka | dp4086

May 9, 2026

Abstract

This project develops a multimodal deep learning system for the *Pixels to Predictions: DL Vision Challenge*, a scientific multiple-choice visual question-answering task. Each example combines an image, question text, answer choices, and optional educational context such as hints and lectures. The goal is to predict the correct answer choice for hidden test examples under strict competition constraints, including offline execution, no external data, free-tier compute, and fewer than 5 million trainable parameters.

The proposed solution fine-tunes SmolVLM-500M-Instruct, a lightweight vision-language model, using parameter-efficient LoRA adaptation. The training pipeline formats each example as an image-text prompt using the model’s chat template and trains only on the answer token through label masking. To handle answer-position imbalance, the pipeline uses weighted sampling during training. During inference, the model avoids free-form generation and instead scores valid answer-choice tokens directly from logits, selecting the most likely option. A cyclic test-time augmentation strategy further reduces option-order bias by rotating answer choices and aggregating predictions.

This approach combines efficient multimodal fine-tuning, controlled multiple-choice decoding, and validation-driven inference selection to produce a reproducible Kaggle submission pipeline under the competition’s compute and academic constraints.

1 Introduction

The *Pixels to Predictions* challenge is a multimodal scientific question-answering competition. Each example contains an image, a question, multiple answer choices, and optional supporting context such as hints, lectures, subject, topic, and skill metadata. The objective is to build a model that uses both the visual input and the textual context to select the correct multiple-choice answer. The final prediction must be a 0-indexed integer corresponding to the selected option, and performance is measured using classification accuracy on hidden test labels.

This task is challenging because it requires both visual interpretation and language reasoning. The model may need to understand scientific diagrams, charts, maps, natural images, or experimental setups, while also using the provided question, choices, and educational context to identify the best answer. Since the output is a single option index, the system must rank candidate answers reliably rather than generate open-ended text.

1.1 Constraints

The competition is governed by strict academic and practical constraints, and we follow all of the specified competition rules in our proposed pipeline:

- Only the provided competition data may be used, including the training, validation, test files, and supplied images.
- External datasets, additional labels, synthetic data from outside corpora, or other pretrained resources are not allowed.
- The only permitted base model is `HuggingFaceTB/SmolVLM-500M-Instruct`.
- Evaluation is conducted offline with no internet access, so the submitted pipeline must run without network connectivity.
- Development and training are limited to free-tier resources such as Google Colab Free/student tier and Kaggle Free tier.
- The total number of trainable parameters must remain below 5 million, including LoRA adapters, classification heads, or any other updated weights.

These constraints make full model fine-tuning impractical and motivate a parameter-efficient approach. In this work, We adapt `SmolVLM-500M-Instruct` using LoRA while keeping the base model mostly frozen. The system formats each example as an image-text multiple-choice prompt, trains only the answer-related parameters under the 5M limit, and produces a valid `submission.csv` file with exactly two columns: `id` and `answer`. The goal is to build a reproducible, offline-compatible, competition-compliant vision-language system for scientific multiple-choice reasoning.

2 Data Exploratory Analysis

The exploratory analysis was performed to understand the dataset structure, answer distribution, question format, metadata coverage, and image diversity. The dataset contains three splits: train, validation, and test. Each sample includes an image, question, answer choices, number of choices, and optional context fields such as hint, lecture, subject, topic, category, and skill.

2.1 Dataset Structure

The EDA showed that this task is not a simple image classification problem. Many questions require combining the image with long textual context and multiple-choice reasoning. **The number of answer choices varies across examples, usually between 2 and 5, so the model and inference pipeline must handle variable option counts instead of assuming four fixed choices.**

2.2 Exploratory Observations

A key observation was **answer-position imbalance**. Some answer indices occur much more frequently than others, which can make the model biased toward common option positions. **This motivated the use of weighted sampling and option-order test-time augmentation.** The dataset also contains diverse subjects, topics, and grade levels, showing that the model must generalize across different scientific and educational domains.

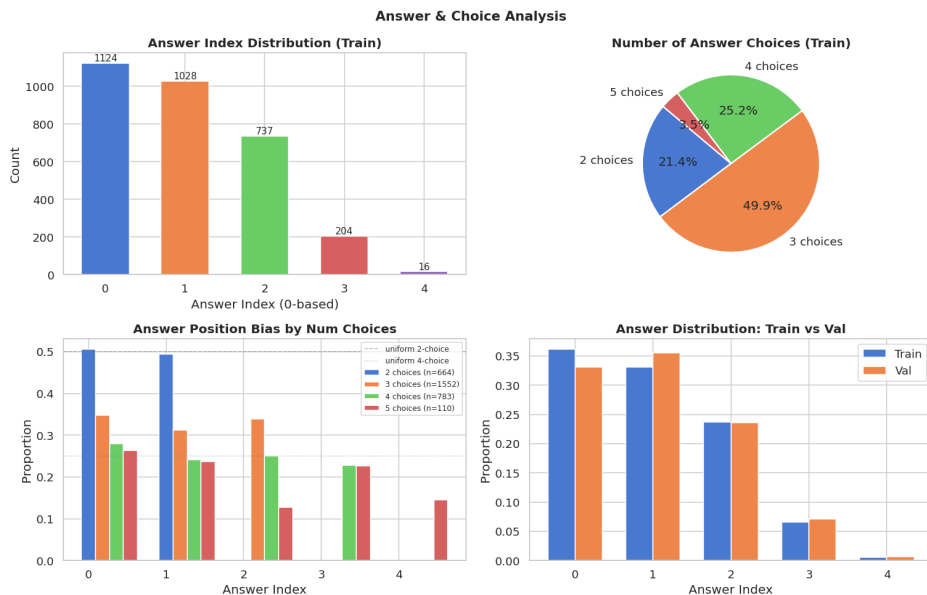


Figure 1: Answer index distribution

The answer distribution is imbalanced across option positions. This can bias the model toward frequently occurring answer indices, so the training pipeline uses weighted sampling and validation-based inference checks

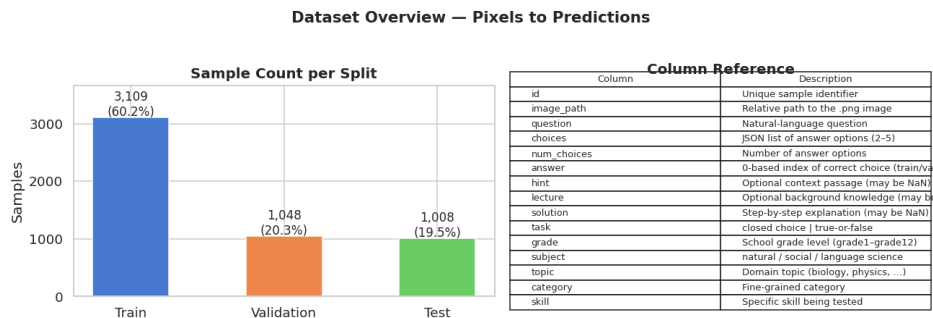


Figure 2: Dataset split sizes

The dataset is divided into train, validation, and test splits. The training and validation sets contain labels, while the test set is unlabeled and used for final leaderboard submission.

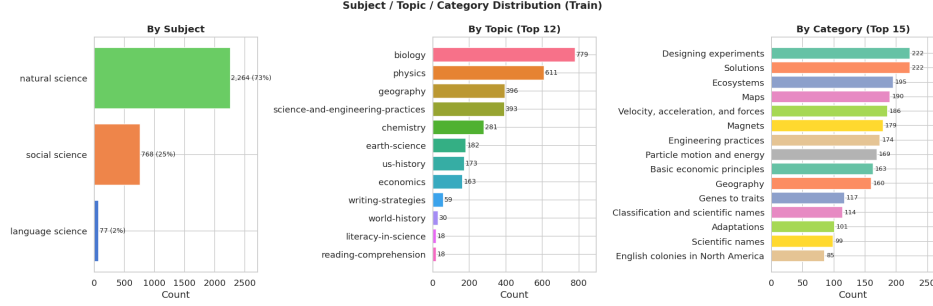


Figure 3: Subject distribution across the dataset

The dataset spans multiple educational domains, with natural science as the dominant subject. This increases the need for a general multimodal model rather than a narrow task-specific classifier.

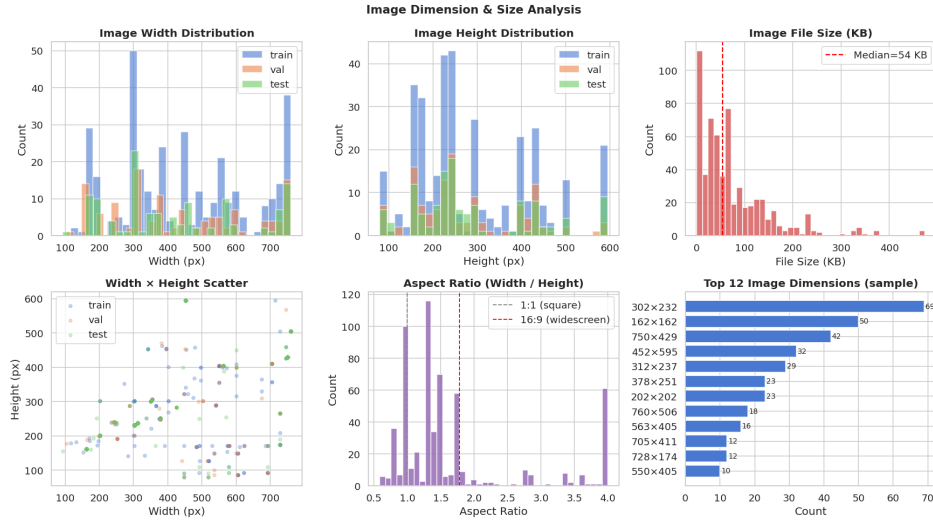


Figure 4: Image dimension distribution across the dataset

The most common image dimensions include repeated sizes such as 302×232 , 162×162 , and 750×429 , suggesting that some images come from common source formats or templates. Overall, this supports the choice of a vision-language model that can handle diverse visual inputs rather than relying on a fixed image classification architecture.

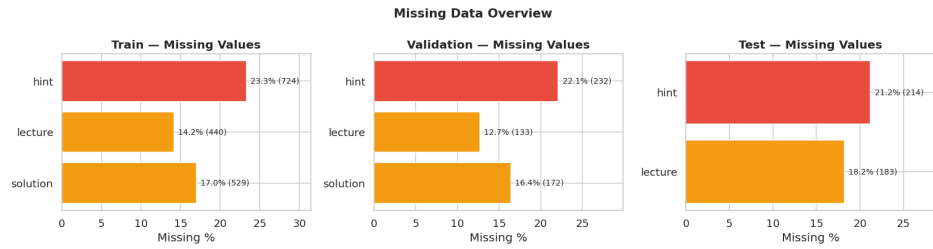


Figure 5: Missing supplementary context fields across dataset splits

Notes on missing columns:

- `hint` — present only when the question has an explanatory passage
- `lecture` — background knowledge text; absent for some questions
- `solution` — full worked solution; absent for some questions

These are supplementary context fields that the VLM can optionally use. None are available in `test.csv` except for the `hint` column.

2.3 Implications for Modeling

The image analysis showed strong visual diversity. Some images are simple diagrams, tables, or symbolic representations, while others are natural science photographs or complex visual scenes. This supports the use of a vision-language model rather than a text-only model. **Overall, the exploratory analysis guided the design of the training and inference pipeline by highlighting the need for multimodal reasoning, flexible handling of answer choices, and bias-aware prediction strategies.**

Final submission note regarding Train-Test-Val split: For the final submission, our primary solution used only the provided training data for model fitting, while the validation dataset was reserved for model selection and validation. We also evaluated a secondary setting that combined the training and validation datasets for the final training run, which produced an additional improvement of about 2%.

3 Methodology

This section describes the end-to-end approach taken to solve the *Pixels to Predictions* multimodal question-answering challenge, covering task formulation, model adaptation, data handling, and key design decisions.

3.1 Task Definition and Objective

The competition presents a multimodal science question-answering task in which each example consists of an image, a natural-language question, an optional context (lecture and hint fields), and a fixed set of answer choices. The model must select the single correct answer from the provided options. **Formally, given an image I , question Q , context C , and choice set $\{c_0, c_1, \dots, c_{n-1}\}$, the objective is to predict the index of the correct choice.**

The evaluation metric is top-1 accuracy — the proportion of test examples for which the model’s top-ranked choice matches the ground-truth answer index. **This metric is aligned with the competition’s official leaderboard scoring and is used both to monitor validation**

performance during training and to select between inference strategies (plain vs. TTA Test-Time Augmentation) for the final submission.

The task is treated as a discriminative generation problem rather than a standard classification problem : the model is prompted to produce the answer letter (A, B, C, ...) and scored by the log-likelihood it assigns to each candidate letter token, enabling evaluation that scales naturally to variable numbers of choices across examples.

3.2 Base Model and Adaptation Setup

SmolVLM-500M-Instruct was specified as the required base model for this challenge, as stated in the competition guidelines. It is a 500-million-parameter vision-language model in the instruction-following family, capable of processing interleaved image and text inputs via a chat-template interface.

Parameter-efficient fine-tuning was applied using Low-Rank Adaptation (LoRA), subject to the competition’s hard constraint of at most 5,000,000 trainable parameters. LoRA decomposes weight updates into low-rank matrices, allowing the backbone weights to remain frozen while a small number of adapter parameters are trained. This is appropriate given the constraint, as it avoids the memory and compute cost of full fine-tuning while still enabling meaningful task adaptation.

LoRA adapters were attached to the four attention projection matrices in every transformer layer: `q_proj`, `k_proj`, `v_proj`, and `o_proj`. These projections govern how the model attends to both the visual tokens and the text tokens, making them the most impactful targets for adapting a pretrained VLM to a new question-answering distribution. **We also experimented with other projection targets, as discussed in the Experiments section.**

Rather than fixing the LoRA rank in advance, the notebook implements a rank-search procedure that iterates over candidate ranks in descending order (32, 24, 16, 12, 8, 4) and selects the largest rank whose resulting trainable parameter count satisfies the 5M cap. This ensures the maximum possible adapter expressiveness within the constraint. **The scaling factor `lora_alpha` is set to $2 \times \text{rank}$, a standard choice that keeps the effective learning rate stable across different rank values.** A dropout of 0.05 is applied to the adapter matrices for regularisation, and bias terms are excluded from training (`bias="none"`) to avoid unnecessary parameter overhead.

The model is loaded in float16 precision, which is both the required and the only dtype used in this solution. `float16` halves memory consumption relative to `float32`, and is appropriate for the Kaggle Tesla T4 $\times 2$ hardware (2×15 GB VRAM) on which the notebook is tuned. Forward and backward passes are wrapped in `torch.amp.autocast(device_type="cuda", dtype=torch.float16)`, which casts compatible operations to `float16` while keeping numerically sensitive operations in `float32`. Because LoRA trains only a small set of adapter parameters and gradient magnitudes were stable throughout our runs, no `GradScaler` was used.

The solution targets the dual-T4 tuning profile specifically. Where both T4 GPUs are available and the batch size permits, the model is wrapped in `torch.nn.DataParallel` to distribute forward and backward passes across both GPUs, effectively doubling the available VRAM and throughput. Training image resolution is fixed at 760 pixels on the long edge for this profile, balancing visual fidelity against VRAM constraints.

Table 1: Hyperparameters for the dual-T4 tuning profile

Hyperparameter	Value
NUM_GPUS	2
MULTI_GPU_MODE	data_parallel
NUM_EPOCHS	3
GLOBAL_BATCH_SIZE	8
PER_GPU_BATCH	2
GRAD_ACCUM	8 (effective batch 32)
LEARNING_RATE	0.0001
WEIGHT_DECAY	0.01
WARMUP_RATIO	0.05
TRAIN_IMAGE_EDGE	760
TRAIN_NUM_WORKERS	2
MEMORY_SAVER_MODE	True

To further manage memory within the 15 GB per-GPU budget, **gradient checkpointing is enabled via `gradient_checkpointing_enable()`**. This trades recomputation of intermediate activations for a significant reduction in memory footprint, making it possible to train with longer sequences and larger image resolutions than would otherwise be feasible on T4 hardware.

3.3 Data Pipeline and Formatting

The competition data consists of three CSV files (train, validation, test) paired with a directory of images. Each CSV row contains an image path, question, choice list (JSON-encoded), optional lecture and hint context fields, and — for labelled splits — an integer answer index.

A custom PyTorch `Dataset` class (`ScienceQADataset`) handles loading and formatting. Each item is constructed as follows:

Image loading: The image is loaded via PIL, converted to RGB, and downsampled so that neither dimension exceeds `IMG_SIZE` pixels, using aspect-ratio-preserving bicubic resampling (thumbnail). `IMG_SIZE` is fixed at 760 pixels for the dual T4 tuning profile used in this solution, balancing visual fidelity against the 15 GB per-GPU VRAM budget of the T4 hardware.

Context handling: The lecture and hint fields are treated as optional context. Both are checked for NaN values before inclusion and concatenated ahead of the question if present.

Prompt formatting: The prompt is assembled using SmolVLM’s chat template via `processor.apply_chat_template`, with a `{"type": "image"}` content block followed by the text. This ensures the image and text tokens are correctly interleaved in the model’s expected format.

Choice presentation: The choices are presented as lettered options (A, B, C, ...) immediately before the **Answer:** cue, following a standard multiple-choice prompt structure.

For training, the collation function (`train_collate`) appends the correct answer letter after the prompt, tokenises the full sequence, and masks all token labels to `-100` except for the single answer-letter token. This confines the cross-entropy loss to the answer position, providing a clean and focused gradient signal.

Removing choice imbalance. Class imbalance in the training set is addressed using a `WeightedRandomSampler`, which assigns each example a sampling weight inversely proportional to its class frequency. For example, if answer position A appears 500 times in the training set and answer position D appears only 100 times, examples with answer D are assigned a sampling weight $5\times$ higher than those with answer A. During each training epoch, the sampler draws examples according to these weights, so every answer position sees roughly equal exposure regardless of its raw frequency in the data. This is achieved entirely through resampling — no examples are duplicated in the dataset itself and no examples are discarded — making it a memory-efficient and lossless approach to handling imbalance.

Per-example choice-order shuffling. Inside the training collator (`train_collate`), each example’s choice list is permuted by an independent random permutation drawn at every iteration, and the gold answer index is remapped to the new position before the supervision letter is appended. This breaks any content–position correlations present in the raw dataset — for example, longer or more specific answers tending to occupy a particular slot. Combined with the class-balanced sampler, the model is exposed to an effectively uniform distribution over correct-letter positions across training, forcing it to learn the content-grounded image–question–answer relationship rather than any positional shortcut.

3.4 Justification of Design Choices

The design choices in this solution were informed by a combination of empirical experimentation, established best practices in parameter-efficient fine-tuning, and published findings on position bias in VLMs.

Single-token training objective. The model is trained using a masked causal language modelling loss. For each training example, the chat-templated prompt ending in **Answer:** is constructed, the correct letter (e.g. B) is appended, and the cross-entropy loss is computed with all token labels masked to -100 except for that single final answer-letter token. The model therefore receives no gradient signal from the prompt, the question text, or the choice list — every backward pass is concentrated exclusively on the token the competition scores, making training directly aligned with the evaluation objective.

Justification of the single-token objective. Several training objectives were evaluated experimentally, including sequence-level generation loss (training the model to produce the full text of the correct answer) and a classification head appended to the final hidden state. **The single-token log-likelihood objective consistently outperformed both alternatives across validation runs.** This outcome is consistent with theoretical expectations: concentrating the cross-entropy loss on one decision-relevant token eliminates gradient noise from tokens in the choice text that carry no discriminative information between options.

LoRA target modules: `q_proj`, `k_proj`, `v_proj`, `o_proj`. LoRA is the standard parameter-efficient alternative, and all four attention projection matrices are targeted rather than only the query and value projections. Adapting `q`, `k`, and `v` together controls how attention scores are computed across both text tokens and image patch tokens, while adapting `o_proj` controls how the attended representation is mixed back into the residual stream. This combination provides the adapter with expressivity over the full attention computation, which is important for cross-modal reasoning where the model must bind text tokens to the correct image regions. MLP layers are deliberately excluded: adding them would force the rank-search to select a very low rank (below 8), reducing representational capacity per matrix and yielding a less expressive adapter overall. We confirmed this during experimentation by adding MLP layers to the target modules alongside `q_proj`, `k_proj`, `v_proj`, and `o_proj`; the rank-search was forced to select rank 4 — the lowest in the search range — and validation accuracy decreased relative to the attention-only configuration. Restricting targets to the four attention projections alone allowed the rank-search to settle at a higher rank within the 5 M cap, producing a more expressive adapter and better validation performance.

Automatic LoRA rank selection. Rather than fixing the LoRA rank in advance, the notebook searches candidate ranks in descending order (32, 24, 16, 12, 8, 4) and **selects the largest rank whose trainable parameter count satisfies the 5 M cap.** A hardcoded rank risks silently violating the cap if the underlying model changes (e.g. a library upgrade alters layer count or parameter naming). The descending search guarantees the maximum adapter capacity within the budget without requiring manual recalibration each time the environment changes.

Cyclic-permutation test-time augmentation (TTA). Vision-language models exhibit measurable position bias: a systematic tendency to favour certain letter positions (most commonly A or B) regardless of content. To counteract this, cyclic-permutation TTA runs N inference passes for each N -choice example, cyclically rotating the choice order by $0, 1, \dots, N - 1$ positions. The resulting probability vectors are un-rotated back to the original choice indices and aggregated. This approach

was validated empirically on the validation set before being applied to test inference; the notebook automatically falls back to plain inference if TTA does not improve validation accuracy. **The technique is research-backed: the first-place solution of the Perception Test Challenge used cyclic-permutation TTA, and it is formalised in the PriDe and BOLD papers as the standard method for neutralising option-order bias in multiple-choice VLMs.**

Confidence-weighted TTA aggregation. Plain cyclic averaging weights every pass equally, including passes where the model assigns nearly uniform probability across all letters — high entropy passes that are essentially noise. Rather than averaging the N probability vectors with equal weight, each pass is weighted by its maximum softmax probability (confidence): $\hat{p} = \sum_k c_k p_k^{(\text{orig})} / \sum_k c_k$, where $c_k = \max(p_k)$ is the confidence of the k -th cyclic shift and $p_k^{(\text{orig})}$ is its probability vector un-rotated to the original choice indices. Confident passes therefore dominate the final prediction while uncertain passes — for example those caused by an unfamiliar choice ordering — contribute proportionally less. This is implemented directly in `predict_dataframe_tta` and empirically reduces TTA’s tendency to wash out a confident plain prediction with several uncertain rotated ones.

Image caching across TTA shifts. An N -choice question requires N inference passes under cyclic-permutation TTA. Without caching, the same JPEG or PNG would be decoded from disk on every pass, introducing pure I/O overhead that scales linearly with the number of choices. **The TTA loop maintains a per-example image_cache dictionary: each image is loaded once on its first pass and reused for all subsequent cyclic shifts, then explicitly closed after the example group is processed to avoid leaking file handles during long inference runs.**

Cosine schedule with linear warmup. A cosine learning rate schedule with a 5% linear warmup is applied (`WARMUP_RATIO = 0.05`). Warmup avoids large gradient steps in the early iterations when the LoRA adapter weights are randomly initialised and the loss surface is steep, and the cosine decay provides smooth annealing toward convergence without requiring manual step-decay tuning.

Empirical plain vs. TTA selection. TTA is not guaranteed to help: a well-calibrated model with low residual position bias may already assign stable probabilities regardless of choice order, in which case TTA averaging introduces noise rather than removing it. **Rather than assuming TTA always helps, the notebook runs both plain and TTA inference on the held-out validation set, computes accuracy for each, and writes the better-performing method to `submission.csv`.** The validation set is never used for gradient updates, so this selection is honest and leak-free. Both `submission_plain.csv` and `submission_tta.csv` are retained on disk to support downstream ensembling with other models.

4 Ablations and Experimental Rigor

The ablation experiments were performed on a reduced experimental setup to keep iteration time manageable under free-tier compute constraints. **Most experiments used approximately 30% of the training data and were evaluated on a fixed subset of 300 validation samples.** This made the comparisons faster while keeping the validation protocol consistent across experiments. The final selected approach was then used to generate predictions for the full test set.

Table 2: Experiment 1

Field	Value
Experiment	Vanilla SmolVLM generation
What Changed	No fine-tuning; model generated the answer directly
Result / Observation	Around 40% on 300 validation samples
Decision	Used as baseline

Table 3: Experiment 2

Field	Value
Experiment	QLoRA with duplicated answer prompt
What Changed	Prompt had duplicated pattern like Answer: Answer: 1
Result / Observation	Poor and confusing outputs, worse than baseline
Decision	Fixed prompt format

Table 4: Experiment 3

Field	Value
Experiment	Numeric choice formatting
What Changed	Choices formatted as 0/1/2/...
Result / Observation	Cleaner for Kaggle answer format
Decision	Useful intermediate setup

Table 5: Experiment 4

Field	Value
Experiment	Letter choice formatting
What Changed	Choices formatted as A/B/C/...
Result / Observation	More natural for language model answer tokens
Decision	Used in final pipeline

Table 6: Experiment 5

Field	Value
Experiment	Full-prompt loss
What Changed	Trained on full prompt plus answer
Result / Observation	Loss decreased, but validation accuracy remained poorer than baseline
Decision	Rejected

Table 7: Experiment 6

Field	Value
Experiment	Answer-only masking
What Changed	Prompt tokens masked with -100; loss only on answer token
Result / Observation	Significant increase in accuracy: 65%
Decision	rank-search procedure

Table 8: Experiment 7

Field	Value
Experiment	Processor-based masking
What Changed	Prompt length computed using full image+text processor instead of tokenizer only
Result / Observation	Verified only answer tokens remained unmasked
Decision	Kept

Table 9: Experiment 8

Field	Value
Experiment	High learning rate
What Changed	Used $1r = 5e-4$
Result / Observation	Unstable and hurt performance
Decision	Rejected

Table 10: Experiment 9

Field	Value
Experiment	Lower learning rate
What Changed	Used $1r = 5e-5$
Result / Observation	More stable; gave strongest early results Final LR: $1e-4$
Decision	Kept

Table 11: Experiment 10

Field	Value
Experiment	QLoRA / LoRA adaptation
What Changed	Trained only LoRA adapters under 5M parameters
Result / Observation	Satisfied compute and rule constraints
Decision	Kept

Table 12: Experiment 11

Field	Value
Experiment	LoRA target modules
What Changed	Applied LoRA to attention projections such as <code>q_proj</code> , <code>k_proj</code> , <code>v_proj</code> , <code>o_proj</code>
Result / Observation	More expressive than minimal q/v-only adaptation
Decision	Kept

Table 13: Experiment 12

Field	Value
Experiment	Answer-index likelihood scoring
What Changed	Scored Answer: 0, Answer: 1, etc.
Result / Observation	Better than free-form generation
Decision	Useful earlier method

Table 14: Experiment 13

Field	Value
Experiment	Direct answer-letter logit scoring
What Changed	Used logits of valid answer letters at answer position
Result / Observation	Avoided invalid generation and directly matched MCQ classification; 3–4% increase in performance
Decision	Final inference method

Table 15: Experiment 14

Field	Value
Experiment	Choice-text likelihood scoring
What Changed	Scored full answer text instead of index/letter
Result / Observation	Weaker early results
Decision	Dropped

Table 16: Experiment 15

Field	Value
Experiment	Class distribution analysis
What Changed	Found strong answer-position imbalance, especially rare class 4
Result / Observation	Explained option-position bias
Decision	Used to guide sampling

Table 17: Experiment 16

Field	Value
Experiment	Heavy balanced oversampling
What Changed	Forced near-equal answer classes
Result / Observation	Sometimes overcorrected and became unstable
Decision	Not final

Table 18: Experiment 17

Field	Value
Experiment	Weighted sampling
What Changed	Used weighted sampling to reduce answer imbalance without manually duplicating too aggressively
Result / Observation	More principled imbalance handling 78% accuracy
Decision	Used in final pipeline

Table 19: Experiment 18

Field	Value
Experiment	Prediction distribution analysis
What Changed	Compared predicted vs. actual answer positions
Result / Observation	Revealed overprediction of common options
Decision	Used for diagnosis

Table 20: Experiment 19

Field	Value
Experiment	Yes/no ranking
What Changed	Candidate answer classified as correct/incorrect
Result / Observation	Poor validation behavior
Decision	Dropped

Table 21: Experiment 20

Field	Value
Experiment	Pairwise ranking loss
What Changed	Tried to make correct answer score better than wrong answer
Result / Observation	Did not beat answer-token/logit approach
Decision	Dropped

Table 22: Experiment 21

Field	Value
Experiment	Cyclic test-time augmentation
What Changed	Rotated answer choices and mapped predictions back to original indices
Result / Observation	Reduced option-order sensitivity
Decision	Used/compared in final pipeline

Table 23: Experiment 22

Field	Value
Experiment	Plain vs. TTA inference comparison
What Changed	Compared normal inference and cyclic TTA on validation
Result / Observation	Selected final submission strategy based on validation accuracy – 82–83%
Decision	Kept

Table 24: Experiment 23

Field	Value
Experiment	Offline two-phase workflow
What Changed	Prep notebook saved model locally; train notebook ran offline
Result / Observation	Satisfied no-internet evaluation constraint
Decision	Kept

Table 25: Experiment 24

Field	Value
Experiment	Submission generation
What Changed	Created <code>submission_plain.csv</code> , <code>submission_tta.csv</code> , and final <code>submission.csv</code>
Result / Observation	Ensured correct Kaggle schema
Decision	Kept

Summary of Experimentation

The experimentation process progressed from a simple baseline toward a controlled multiple-choice inference pipeline. The initial baseline used vanilla SmolVLM generation without fine-tuning and achieved around **40% accuracy** on a 300-sample validation subset. Early QLoRA attempts revealed prompt-formatting issues, especially duplicated answer fields, and showed that training on the full prompt was not effective even when training loss decreased.

The first major improvement came from changing the training objective to **answer-only supervision**. Prompt tokens were masked with -100, so the model optimized only the final answer token. This made the loss more aligned with the competition objective. A further masking fix used the full image-text processor to calculate prompt length, ensuring that only the answer token remained unmasked.

Several inference strategies were compared. Free-form generation was less reliable because it could produce extra or invalid text. Answer-index likelihood scoring improved reliability by scoring valid options directly. Later, the final pipeline moved to **direct answer-letter logit scoring**, where only valid letter tokens such as A, B, C, and D were considered at the answer position. This better matched multiple-choice classification.

The experiments also showed that answer-position imbalance was a major issue. EDA revealed that some answer positions were much rarer than others, especially higher-index choices. Heavy manual oversampling sometimes overcorrected and became unstable, while weighted sampling provided a more controlled way to reduce imbalance. Prediction-distribution analysis was used to verify whether the model was overpredicting common answer positions.

Alternative formulations were also tested, including choice-text likelihood scoring, yes/no ranking, and pairwise ranking loss. These approaches were theoretically reasonable but did not outperform the answer-token/logit scoring pipeline in validation experiments. The final method therefore kept the strongest components: LoRA adaptation, answer-token masking, weighted sampling, direct answer-letter logit scoring, and cyclic test-time augmentation.

Overall, the ablations showed that the largest gains came from aligning both training and inference with the multiple-choice objective. The final system avoids open-ended generation, limits predictions to valid answer options, handles class imbalance through sampling, and reduces option-order sensitivity through cyclic TTA.

Error Analysis and Limitations

The main failure mode observed during validation was **option-position bias**. The model often preferred common answer positions and struggled with rare positions, especially higher-index choices. EDA showed that the training labels were highly imbalanced, with some answer indices appearing far more often than others. Weighted sampling helped reduce this effect, but it did not remove the bias completely.

A second limitation is the model’s sensitivity to **answer ordering**. Since multiple-choice options are arbitrary, the same correct answer can appear at different positions. This motivated cyclic test-time augmentation, where answer choices are rotated and predictions are mapped back to the original indices. However, TTA increases inference cost because each example must be scored multiple times.

Some errors also came from **visual complexity**. The dataset includes diagrams, charts, maps, scientific illustrations, and natural images. Questions requiring small visual details or text inside images are harder, especially after resizing images to a fixed input size. In these cases, the model may rely more on the textual context than the image.

Another limitation is the presence of **long context fields** such as lectures and hints. These fields can help reasoning, but they also make prompts longer and may distract the model if the useful evidence is buried inside irrelevant text. The model must learn which parts of the context are important for the question.

The experiments were also limited by compute constraints. Most ablations were run on approximately **30% of the training data** and evaluated on **300 validation samples** to allow faster iteration. This made experimentation practical, but some results may differ when training on the full dataset or evaluating on the complete validation set.

Finally, the solution uses LoRA rather than full fine-tuning. This keeps the trainable parameter count within the competition limit, but it also limits how much the model can adapt to the dataset. Larger models, full fine-tuning, or stronger ensembling may improve performance, but they were not feasible under the competition constraints.

Metric

Accuracy was used as the primary evaluation metric because the competition is a multiple-choice classification task. For each validation or test example, the model predicts one answer option, and the prediction is counted as correct only if it exactly matches the ground-truth answer index. During experimentation, most runs were compared on a fixed subset of 300 validation samples to keep results consistent and iteration time manageable. This made it possible to compare prompt formats, masking strategies, LoRA settings, sampling methods, and inference techniques under the same evaluation setup.

Limitations

The main limitation of the experiments was that many ablations were performed on only 30% of the training data and 300 validation samples due to compute limits, so the results may not fully represent full-dataset behavior. The dataset also had strong answer-position imbalance, which made the model biased toward frequent option positions and required weighted sampling. Some questions depended on small visual details, dense diagrams, or long lecture context, which may be difficult for SmolVLM after image resizing and prompt truncation. Finally, LoRA adaptation kept the solution within the 5M trainable-parameter limit, but it also restricted how much the base model could adapt compared with full fine-tuning.

Selected References

The backbone model used in this project is grounded in Idefics3, the work of Laurençon et al. [4], which explains the architecture and training design behind the SmolVLM family. That paper provides the conceptual basis for using a compact vision-language model with an efficient vision

encoder and decoder-style language backbone in resource-constrained settings.

The fine-tuning strategy follows LoRA, introduced by Hu et al. [1]. Their work shows how low-rank adapter updates can preserve most pretrained weights while still enabling effective task adaptation with a small number of trainable parameters, which directly supports the competition constraint of staying below the trainable-parameter limit.

The concern about option-position bias is consistent with the findings of Ko et al. [3], who show that large language models are often unreliable multiple-choice selectors because they can become biased toward particular answer positions. This observation aligns with the behavior seen during validation and motivates the use of bias-reduction strategies.

The cyclic-permutation inference strategy is also supported by the multimodal bias analysis of Jiang et al. [2], whose work studies modality- and position-related failure modes in multimodal large language models. Their results give theoretical support for permutation-based debiasing and ensemble-style inference in multiple-choice settings.

Finally, the broader multimodal question-answering setup is closely related to instruction-tuned vision-language systems such as LLaVA [5]. That line of work helps contextualize the present pipeline as a lightweight, competition-constrained adaptation of the now-standard image-plus-text instruction-following paradigm.

References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022.
- [2] Zhen Jiang et al. Mitigating modality prior-induced hallucinations in multimodal large language models via deciphering attention causality. *arXiv preprint arXiv:2410.04780*, 2024.
- [3] Miyoung Ko et al. Large language models are not robust multiple choice selectors. *arXiv preprint arXiv:2309.03882*, 2023.
- [4] Hugo Laurençon et al. Idefics3: Building and better understanding vision-language models. *arXiv preprint arXiv:2408.12637*, 2024.
- [5] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 2023.